

مستند فنی فارسی

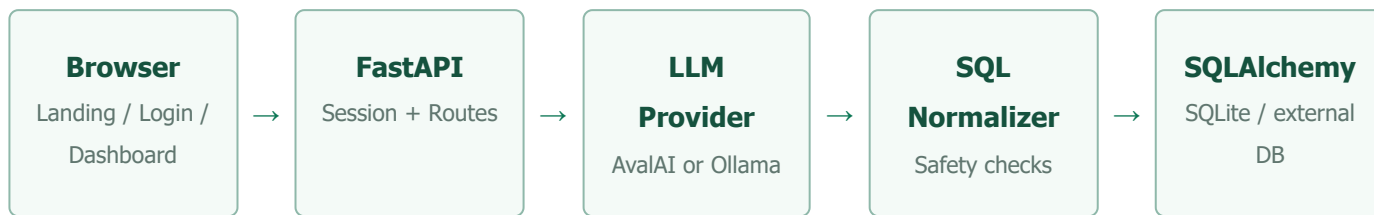
برنچ newDashboard2

شرح معماری، جریان اجرا، API ها، مدل داده، تمام فایل های tracked و
منطق کدهای اصلی

تاریخ commit 2026-07-15T18:09:00+03:30	Commit 7643d0fd45
تعداد فایل ها 30 فایل	عنوان Change first page of DashboardCreate

منبع این سند مستقیماً از Git object های branch خوانده شده است، نه از تغییرات ثبت نشده working tree. مفادیر secret عمداً در سند بازنشر نشده اند.

خلاصه سامانه



مسیر اصلی سؤال: مرورگر سؤال و تنظیمات زبان/مدل را به `/api/chat` می‌فرستد؛ مدل یک JSON شامل SQL و توضیح می‌سازد؛ SQL اصلاح و ایمن‌سازی می‌شود؛ با Pandas/SQLAlchemy اجرا می‌گردد؛ نتیجه و زمان هر مرحله به رابط برمی‌گردد. تحلیل هوشمند عمداً در درخواست جداگانه `/api/analyze` و با اقدام کاربر اجرا می‌شود.

- **هدف:** پرسش فارسی/انگلیسی درباره تراکنش‌ها، تبدیل به SQL و نمایش جدول/نمودار/تحلیل.
- **Backend:** FastAPI، Pydantic، SQLAlchemy و Pandas.
- **مدل زبانی:** AvalAI ابری یا Ollama محلی با interface یکسان.
- **Frontend:** HTML/CSS/JavaScript خالص و Chart.js.
- **هویت:** session امضاشده، حساب `app_users` و نقش `admin/user`.
- **داده:** star schema آزمایشی PSP روی SQLite با قابلیت تغییر `DATABASE_URL`.

مسیرهای HTTP و سطح دسترسی

مسیر	دسترسی	وظیفه
GET /	عمومی	نمایش صفحه معرفی static/first.html
GET /login	عمومی	نمایش فرم ورود؛ کاربر واردشده به /app منتقل می‌شود.
POST /api/login	عمومی	بررسی چک‌باکس کپچا، نام کاربری و گذرواژه؛ ایجاد نشست.
POST /api/logout	واردشده	پاک‌کردن کامل داده‌های نشست.
GET /api/me	واردشده	اطلاعات کاربر جاری به‌همراه تنظیمات مدل و زبان.
GET /app	واردشده	نمایش داشبورد گفتگو از static/index.html.
GET /users	مدیر	صفحه مدیریت کاربران.
GET /api/users	مدیر	فهرست حساب‌های برنامه.
POST /api/users	مدیر	ساخت حساب و هش کردن گذرواژه.
PUT /api/users/{id}	مدیر	ویرایش حساب، نقش، وضعیت و گذرواژه.
DELETE /api/users/{id}	مدیر	حذف حساب با محافظت از آخرین مدیر و حذف خود.
GET /api/health	واردشده	بررسی پایگاه داده، Ollama و AvalAI.
GET /api/avalai/credit	واردشده	پروکسی سمت سرور برای مانده اعتبار AvalAI.
POST /api/chat	واردشده	تبدیل سؤال به SQL، نرمال‌سازی، کنترل ایمنی، اجرا و ثبت تاریخچه.
POST /api/analyze	واردشده	تحلیل اختیاری حداکثر ۲۰ ردیف اول نتیجه توسط مدل انتخاب‌شده.
GET /api/history	واردشده	۵۰ پیام اخیر همان نام کاربری.
GET /api/preferences	واردشده	خواندن مدل و زبان ذخیره‌شده کاربر.
PUT /api/preferences	واردشده	ذخیره provider و language معتبر.

مدل داده

جدول	نقش	ستون‌ها/رابطه‌های مهم
app_users	حساب‌های ورود	id, username, password_hash, mobile, display_name, role, is_active, زمان‌ها
dim_date	بعد تاریخ	date_key به شکل YYYYMMDD, full_date, روز/ماه/سال, تعطیلی و آخر هفته
dim_category	بعد صنف	شناسه و عنوان فارسی ۱۵ صنف استاندارد
dim_customer	مشتری قراردادی PSP	نام, access_key نمایشی و نقش customer/admin: با حساب app_users متفاوت است.
dim_merchant	پذیرنده	نام/کد, MCC, صنف, شهر, مالک مشتری و وضعیت فعال
dim_terminal	پایانه	سریال, پذیرنده, نوع POS/mPOS/Online, تاریخ نصب و وضعیت
fact_transactions	فکت تراکنش	تاریخ, پایانه, پذیرنده, مبلغ, وضعیت, PAN ماسک‌شده, پاسخ, تسویه و کانال

قاعده حیاتی تاریخ: fact_transactions.date_key یک عدد YYYYMMDD است؛ برای فیلتر تقویمی باید join dim_date شود و از full_date استفاده گردد. هم prompt مدل و هم sql_safety این قاعده را تقویت می‌کنند.

فایل‌های Backend

خط 557

app.py

مرکز اتصال تمام اجزای برنامه و تعریف middleware، FastAPI، مدل‌های ورودی و endpointها.

- دو provider با مجموعه VALID_PROVIDERS کنترل می‌شوند و get_llm_client ماژول AvalAI یا Ollama را برمی‌گرداند.
- ترتیب middlewareها طوری است که SessionMiddleware بیرونی باشد و نشست قبل از RequireLoginMiddleware در دسترس قرار گیرد.
- رشته SCHEMA_DESCRIPTION ساختار دقیق جداول و قواعد تاریخ/وضعیت را به مدل می‌دهد تا SQL سازگار با SQLite تولید شود.
- در /api/chat زمان تولید SQL، نرمال‌سازی، کنترل ایمنی، اجرا و ذخیره حافظه جداگانه اندازه‌گیری می‌شود.
- اجرای SQL با pandas.read_sql_query(text(...)) انجام و DataFrame به رکورد JSON تبدیل می‌شود.
- مدیریت کاربران فقط با نقش admin در دسترس است و حذف خود مدیر در route نیز مسدود می‌شود.

get_llm_client(provider) انتخاب پیاده‌سازی مشترک generate_sql/generate_analysis.

RequireLoginMiddleware.dispatch تفکیک مسیرهای عمومی، API بدون نشست و صفحات نیازمند ورود.

_startup_init_users ساخت جدول app_users و مدیر آغازین هنگام startup.

require_admin تولید پاسخ 401 یا 403 برای عملیات مدیریتی.

chat خط لوله اصلی Text-to-SQL و اجرای گزارش.

analyze تحلیل ثانویه و ثبت تحلیل روی آخرین پیام متناظر.

get_history /
get_preferences /
put_preferences APIهای حافظه و تنظیمات شخصی.

تنظیم مرکزی SQLAlchemy؛ تنها نقطه انتخاب SQLite یا پایگاه داده خارجی.

- DATABASE_URL از environment خوانده می‌شود و در حالت پیش‌فرض به `psp_bi_mock.db` اشاره دارد.
- برای SQLite گزینه `check_same_thread=False` فعال است؛ برای سایر DBها `connect_args` خالی است.
- `pool_pre_ping=True` اتصال مرده را پیش از استفاده تشخیص می‌دهد.

engine موتور مشترک SQLAlchemy.

SessionLocal کارخانه نشست با autocommit/autoflush خاموش.

Base پایه declarative مدل‌های ORM.

get_db dependency مولد که نشست را در finally می‌بندد.

تعریف star schema آزمایشی و ساخت داده واقع‌گرایانه تراکنش‌های پرداخت ایران.

- شش مدل داده BI، رابطه‌های ۱۵، ORM صنف، MCC‌های متناظر، پذیرندگان فارسی، شهرها و مشتریان نمونه را تعریف می‌کند.
- seed ثابت ۴۲ باعث بازتولیدپذیری داده‌های تصادفی می‌شود.
- `build_database` فایل/جدول را بازسازی، ابعاد را درج و تراکنش‌ها را با توزیع مبلغ، وضعیت و کانال تولید می‌کند.
- اجرای دوباره این فایل مخرب است: دیتابیس mock را از نو می‌سازد.

DimDate / DimCategory ابعاد تقویم و صنف.

DimCustomer / DimMerchant / DimTerminal مالکیت تجاری، پذیرنده و پایانه.

FactTransaction جدول رخدادهای مالی.

weighted_choice انتخاب احتمالی وزن‌دار برای داده طبیعی‌تر.

mask_pan تولید شماره کارت ماسک‌شده نمایشی.

build_database(n_transactions=1000) ساخت کامل schema و داده نمونه.

توابع کوچک و متمرکز برای احراز هویت مبتنی بر session.

- کلیدهای user_id، role، username و display_name را در کوکی نشست امضا شده نگه می‌دارد.
- اعتبارسنجی واقعی گذرواژه به users.py واگذار می‌شود.
- SESSION_SECRET قابل تنظیم با environment است.

verify_credentials روی wrapper authenticate_user.

set_session_user نوشتن داده عمومی کاربر در نشست.

is_authenticated / is_admin کنترل سریع وضعیت و نقش.

current_user ساخت dict عمومی از session.

مدل و عملیات CRUD حساب‌های ورود برنامه؛ مستقل از dim_customer تحلیلی.

- گذرواژه با salt، PBKDF2-HMAC-SHA256 تصادفی و ۱۲۰ هزار iteration ذخیره می‌شود.
- مقایسه digest با hmac.compare_digest انجام می‌شود.
- مدیر bootstrap از environment یا مقادیر پیش‌فرض ساخته و همیشه فعال/admin نگه داشته می‌شود.
- کد از حذف، غیرفعال‌سازی یا تنزل آخرین مدیر فعال جلوگیری می‌کند.

AppUser مدل ORM حساب، نقش و وضعیت.

hash_password /
verify_password هاش و بررسی گذرواژه.

init_users_table /
seed_bootstrap_admin راه‌اندازی اولیه حساب‌ها.

authenticate_user بررسی وجود، فعال بودن و رمز.

list_users / create_user /
update_user / delete_user عملیات مدیریت کاربران با کنترل‌های کسب‌وکار.

حافظه گفتگو و ترجیحات جداگانه برای هر username با persistence در history.json.

- ساختار دیسک شامل preferences، users و messages است.
- Lock درون‌پرده‌ای از هم‌زمانی threadها هنگام خواندن/نوشتن جلوگیری می‌کند.
- فرمت قدیمی که یک list ساده بوده به bucket کاربر anonymous مهاجرت داده می‌شود.
- history.json در Git نیست و به‌وسیله gitignore حذف شده است.

`_ensure_user` ساخت bucket و تکمیل preference‌های پیش‌فرض.

`add_message / get_context` ثبت پیام و دریافت n پیام پایانی.

`update_last_analysis` افزودن تحلیل به آخرین سؤال مطابق.

`get_preferences / set_preferences` مدیریت provider و language.

`save_to_disk / load_from_disk` serialization فارسی با JSON و بازیابی مقاوم به خطا.

لایه اصلاح خطاهای رایج مدل و رد SQLهای غیرخواندنی پیش از اجرا.

- عبارتهای date_key را به join با dim_date و full_date تبدیل می‌کند.
- modifierهای نادرست ماه و حروف وضعیت Active/Inactive را برای SQLite اصلاح می‌کند.
- LIMIT ناخواسته مدل را فقط وقتی کاربر تعداد مشخص نخواست حذف می‌کند.
- فقط یک SELECT یا WITH را می‌پذیرد، semicolon میانی و keywordهای مخرب را رد می‌کند و LIMIT بیش از ۱۰۰ هزار را کاهش می‌دهد.

fix_date_key_sql بازنویسی تاریخ integer به بعد تاریخ.

fix_sqlite_date_modifiers اصلاح first/last day of month.

fix_status_literals نرمال‌سازی active/inactive.

normalize_generated_sql اجرای زنجیره تمام fixها.

user_requested_row_limit / strip_unrequested_limit تشخیص درخواست تعداد به فارسی/انگلیسی.

validate_select_only دروازه نهایی SQL خواندنی.

کلاینت HTTP مدل محلی Ollama با قرارداد عمومی مشترک با AvalAI.

- به 127.0.0.1:11434 متصل می‌شود و trust_env را خاموش می‌کند تا proxy سیستم در Windows دخالت نکند.
- خروجی SQL باید JSON با کلیدهای sql و explanation باشد.
- زنجیره parsing خروجی مستقیم، code fence، اولین بلوک آکولادی، quoteهای تکی، comma پایانی و newline داخل string را تحمل می‌کند.
- تولید تحلیل متن آزاد با temperature بالاتر و زبان فارسی/انگلیسی انجام می‌شود.

_post فراخوانی /api/generate/ و تبدیل خطاهای شبکه به OllamaError.

_escape_literal_control_characters_in_strings تعمیر newline/tab غیرمجاز در JSON مدل.

_try_parse / _extract_json استخراج مقاوم JSON.

generate_sql ساخت prompt دقیق SQLite و بازگرداندن dict.

generate_analysis تحلیل ۲۰ ردیف نمونه به زبان انتخابی.

کلاینت API سازگار با OpenAI برای AvalAI با همان interface مازول Ollama.

- مدل از AVALAI_MODEL و کلید از AVALAI_API_KEY خوانده می‌شود.
- درخواست chat/completions با Bearer token و timeout شنیت‌ثانی‌ای ارسال می‌شود.
- prompt و parser از ollama_client بازاستفاده می‌شوند تا رفتار دو همسان بماند.
- تابع check_credit به endpoint حساب AvalAI متصل می‌شود؛ مقدار کلید در مرورگر ارسال نمی‌شود.

_chat فراخوانی chat/completions و مدیریت Connection/Timeout/HTTP/Format.

check_credit دریافت اعتبار حساب از سمت سرور.

generate_sql تولید SQL JSON با قواعد schema.

generate_analysis تحلیل متنی داده نمونه.

نقطه ورود میزبانی cPanel/Phusion Passenger

- دایرکتوری برنامه را به sys.path اضافه می‌کند.
- FastAPI که ASGI است با `a2wsgi.ASGIMiddleware` به application سازگار با WSGI تبدیل می‌شود.

application شیء entry point مورد انتظار Passenger.

فایل‌های Frontend

خط 701

static/first.html

صفحه عمومی معرفی Rayamate که route ریشه آن را نمایش می‌دهد.

- Iconify، Tailwind CDN و Google Fonts را از اینترنت می‌گیرد و CSS تکمیلی را inline نگه می‌دارد.
- بخش‌های hero، آمار، نمایش محصول، قابلیت‌ها، CTA و footer دارد.
- Canvas ذرات متحرک و خطوط ارتباطی، شمارنده آماری با IntersectionObserver، اسکرول نرم و tilt کارت‌ها با inline JavaScript پیاده شده‌اند.
- لینک‌های ورود هنگام اجرا با hostname جاری و پورت ۸۰۰۰ بازنویسی می‌شوند.

مدل هر ذره پس‌زمینه. `Particle.reset/update/draw`

انطباق canvas و حلقه انیمیشن. `resizeCanvas /
animateParticles`

افزایش نرم اعداد هنگام ورود به viewport. `animateStats`

ساخت URL صفحه login. `setAppLinks IIFE`

خط 629

static/login.html

صفحه ورود راست‌به‌چپ با CSS کامل inline و اتصال به login.js.

- طرح دو ستونه معرفی محصول و فرم ورود؛ در موبایل تک‌ستونه می‌شود.
- ورودی نام کاربری، رمز، remember me، چک‌باکس «من ربات نیستم»، نمایش رمز و پیام خطا دارد.
- بازیابی رمز و ورود سازمانی در UI حضور دارند ولی در این نسخه فقط پیام «به‌زودی» نشان می‌دهند.

کنترل رفتار فرم ورود در مرورگر.

- فقط username را در localStorage نگه می‌دارد و هرگز password را ذخیره نمی‌کند.
- قبل از ارسال، خالی نبودن فیلدها و فعال بودن checkbox کپچا را بررسی می‌کند.
- درخواست JSON به /api/login می‌فرستد و پس از موفقیت به /app می‌رود.

submit handler اعتبارسنجی، disable موقت دکمه و مدیریت پاسخ/خطا.

toggle password handler تغییر type بین password و text.

showError نمایش پیام خطای فارسی.

پوسته داشبورد احرازشده و فایل واقعی سروشده در route /app.

- header شامل عنوان، انتخاب AvalAI/Ollama، زبان، لینک مدیریت کاربران برای admin و خروج است.
- ستون گفتگو شامل welcome، chat-area، textarea و Send است.
- sidebar شامل تاریخچه، چهار اقدام سریع، تنظیمات مدل/زبان و درباره دستیار است.
- Chart.js محلی و static/script.js در پایان صفحه بارگذاری می‌شوند.

منطق اصلی کلاینت داشبورد؛ بدون framework و مبتنی بر DOM API.

- در startup از api/me/ هویت و preference ها را می‌خواند؛ لینک کاربران را فقط برای admin آشکار می‌کند.
- ارسال سؤال، loading strip، خطا، SQL جمع‌شونده، توضیح، جدول صفحه‌بندی‌شده، نمودار و تحلیل را مدیریت می‌کند.
- جدول اندازه صفحه ۵۰ تا ۱۰۰۰ دارد و برچسب ستون‌های رایج را فارسی می‌کند.
- برای نمودار، ستون عددی و برچسب را هوشمند انتخاب و bar/line/pie/doughnut/horizontal را پشتیبانی می‌کند.
- متن تحلیل مدل escape می‌شود و سپس /heading/list/عددها به HTML کنترل‌شده تبدیل می‌شوند.

همگام‌سازی state و دکمه‌ها. `applyLanguageUI /
applyProviderUI`

ارتباط session-aware با backend. `apiFetch / logout /
savePreferences`

نمایش معکوس تاریخچه و بازگرداندن سؤال به composer. `loadHistorySidebar`

نمایش اختیاری مانده اعتبار کنار provider. `refreshAvalaiCredit`

چرخه کامل هر turn گفتگو. `sendMessage / renderError /
renderResult`

نمایش سهم زمانی مراحل backend. `renderTimingsSection`

فراخوانی جداگانه /api/analyze/. `requestAnalysis`

ساخت امن و صفحه‌بندی جدول. `buildTableHtml /
mountPaginatedTable`

استنباط ساختار نمودار. `pickNumericColumn /
pickLabelColumn /
prepareChartRows`

ساخت و تعویض نمودار Chart.js. `createChartInstance /
maybeRenderChart`

ایمنی خروجی و پشتیبانی دوجته. `escapeHtml /
formatAnalysisHtml /
isRtlText`

سامانه طراحی مشترک داشبورد و صفحه کاربران.

- متغیرهای رنگ، کاغذ، accent، danger، فونت sans/mono/RTL و قواعد پایه را تعریف می‌کند.
- layout داشبورد، sidebar قابل جمع‌شدن، کارت‌ها، تاریخچه، composer و message turn ها را پوشش می‌دهد.
- استایل SQL، result strip، جدول pagination، نمودار، تحلیل، loading، timing bar و responsive breakpoints را دارد.
- قواعد [dir=rtl] برای جای‌گذاری دکمه‌ها و متن فارسی استفاده شده‌اند.

صفحه CRUD کاربران برای مدیر.

- style اختصاصی صفحه را inline و style.css مشترک را جداگانه بارگذاری می‌کند.
- جدول کاربران، حالت خالی، پیام عملیات و modal افزودن/ویرایش دارد.
- فرم شامل نام نمایشی، username، موبایل، role، password و وضعیت فعال است.

اتصال UI مدیریت کاربران به API های admin.

- wrapper مشترک api پاسخ‌های 401/403 را مدیریت و برای 401 به هدایت می‌کند.
- ردیف‌های جدول با escapeHtml ساخته و دکمه ویرایش/حذف به هر کاربر متصل می‌شوند.
- در create رمز الزامی و در edit اختیاری است؛ modal بیرونی قابل بستن است.
- ابتدا api/me/ نقش را کنترل و سپس کاربران را بارگذاری می‌کند.

api fetch و تبدیل پاسخ خطا به exception.

openModal / closeModal حالت create/edit فرم.

renderUsers / loadUsers رندر و دریافت فهرست.

removeUser تأیید و DELETE.

form submit handler PUT یا POST بر اساس user-id.

نسخه minified و vendored کتابخانه Chart.js برای ترسیم نمودار بدون وابستگی runtime به CDN.

- کد third-party تولیدشده است و نگهداری دستی آن توصیه نمی‌شود.
- از طریق global به نام `Chart` در `static/script.js` مصرف می‌شود.

دارایی تصویری دودویی مربوط به الگو/مرجع طراحی صفحه ورود.

- فایل PNG کد اجرایی ندارد؛ در مستند فقط نقش و اندازه آن ثبت می‌شود.

فایل‌های پیکربندی، مستند و legacy

خط 17

Dockerfile

ساخت image سبک Python 3.11 برای اجرا در container/Hugging Face Spaces.

- dependency را پیش از COPY کل پروژه نصب می‌کند تا cache لایه‌ها بهتر استفاده شود.
- پورت ۷۸۶۰ را expose می‌کند؛ در startup اگر DB وجود نداشته باشد db_mock.py را اجرا می‌کند و سپس Uvicorn را بالا می‌آورد.

خط 8

requirements.txt

فهرست dependency های runtime پایتون.

- FastAPI/Uvicorn برای وب، SQLAlchemy/Pandas برای داده، Requests برای مدل‌ها، python-multipart و itsdangerous برای وب و a2wsgi برای Passenger.
- بعضی package ها pin نشده‌اند؛ build آینده می‌تواند نسخه متفاوتی دریافت کند.

خط 178

README.md

راهنمای پروژه، معماری اولیه، setup و توضیح تعویض پایگاه داده.

- بخش‌هایی از README با کد جدید همگام نیست؛ هنوز auth مبتنی بر access_key و endpoint های قدیمی را توصیف می‌کند، در حالی که branch حساب app_users و session دارد.
- راهنمای Ollama، ساخت DB، اجرای Uvicorn و تنظیم DATABASE_URL همچنان مفید است.

خط 28

.gitignore

حذف فایل‌های Python، محیط مجازی، archive، DB/history/env و تنظیمات IDE از Git.

- به‌درستی .env و *.db را حذف می‌کند؛ اما secret ی که مستقیماً در سورس نوشته شود با .gitignore محافظت نمی‌شود.

cookie jar با قالب Netscape که curl ساخته است.

- در commit حاضر فقط header دارد و cookie فعال ذخیره نکرده است.
- اصولاً فایل‌های cookie واقعی نباید commit شوند، چون ممکن است session حساس داشته باشند.

اسکرپت ساخت خودکار ارائه فارسی PowerPoint درباره پروژه.

- با python-pptx شکل‌ها، textbox‌ها، bullet‌ها، footer و ۱۱ اسلاید را تولید می‌کند.
- فونت فارسی، جهت و رنگ سازمانی را روی run‌های PowerPoint اعمال می‌کند.
- dependency python-pptx در requirements.txt حاضر نیست و برای بازتولید ارائه باید جداگانه نصب شود.

تنظیم فونت و پاراگراف فارسی. `set_run_font / set_textbox`

ساخت عناصر هندسی. `add_rect / add_round_rect`

لیست و شماره صفحه. `add_bullets / footer`

تولید کل deck و ذخیره فایل. `build`

خروجی دودویی ارائه ساخته‌شده برای معرفی معماری و قابلیت‌ها.

- کد نیست و محتوای داخلی OOXML در این مستند line-by-line بررسی نمی‌شود؛ منبع قابل نگهداری آن `create_presentation.py` است.

نسخه قدیمی/مرجع پوسته گفتگو در ریشه repository.

- FastAPI این فایل را سرو نمی‌کند؛ route فعال به static/index.html اشاره دارد.
- نسخه ریشه sidebar، نشست و مدیریت کاربران جدید را ندارد و احتمالاً artifact باقی‌مانده از طراحی قبلی است.

JavaScript قدیمی رابط access_key در ریشه.

- DOM‌هایی مانند login-gate و API‌هایی مانند api/customers/ را انتظار دارد که با app.py فعلی هماهنگ نیستند.
- در صفحات فعال load نمی‌شود و بهتر است در آینده حذف، archive یا صریحاً به‌عنوان legacy علامت‌گذاری شود.

CSS قدیمی رابط تک‌ستونه در ریشه.

- صفحات فعال /static/style.css را می‌خوانند؛ این فایل توسط FastAPI mount نشده است.
- توکن‌های visual identity اولیه و login-gate قدیمی را نگه می‌دارد.

جریان‌های اجرایی مهم

ورود

login.js → POST /api/login → users.authenticate_user → verify_password → set_session_user → cookie session → redirect /app

گفتگو

sendMessage → POST /api/chat → provider.generate_sql → strip/fix/validate SQL → read_sql_query → memory.add_message → renderResult

تحلیل

Analyze button → POST /api/analyze → provider.generate_analysis(first 20 rows) → memory.update_last_analysis → formatAnalysisHtml

مدیریت کاربر

/users → require_admin → users.js CRUD → /api/users → users.py → SQLAlchemy app_users

تغییر تنظیمات

/api/me بارگذاری مجدد از → MemoryManager per-user → PUT /api/preferences → دکمه مدل/زبان

ریسک‌ها و پیشنهادهای فنی

بالا — credential‌های پیش‌فرض

نام کاربری/گذرواژه مدیر bootstrap و SESSION_SECRET دارای default ثابت هستند. در production باید startup بدون environment امن fail شود و secret قوی تزریق گردد.

بحرانی — کلید API در سورس

در commit این branch برای AvalAI یک مقدار پیش‌فرض واقعی‌نما داخل کد وجود دارد. مقدار در این سند عمداً بازنشر نشده است. کلید باید فوراً revoke/rotate شود و فقط از secret manager یا environment دریافت گردد؛ default باید خالی باشد.

متوسط — CSRF و cookie

عملیات POST/PUT/DELETE token ضد-CSRF ندارند. تنظیم SameSite=Lax secure/https-only کوکی نیز صریح نیست. کمک می‌کند اما جایگزین کامل CSRF protection نیست.

متوسط — کپچای نمایشی

captcha فقط یک boolean ارسالی از مرورگر است و اثبات ضدریات نیست. برای محیط عمومی باید سرویس captcha واقعی و rate limiting اضافه شود.

متوسط — نبود row-level scope

README از محدودسازی داده مشتری صحبت می‌کند، اما app.py فعلی سؤال کاربر واردشده را بدون scope مبتنی بر مشتری روی DB اجرا می‌کند. app_users نیز به dim_customer متصل نیست.

متوسط — اعتبارسنجی SQL regex

لایه فعلی برای prototype مفید است ولی parser کامل SQL نیست. اجازه WITH/SELECT، نبود timeout/query-cost و حذف LIMIT می‌تواند query سنگین ایجاد کند. کاربر DB باید read-only و محدود باشد.

پایین — dependency و CDN

چند dependency بدون version pin و صفحه اول وابسته به CDN است. برای build تکرارپذیر و شبکه بسته، lockfile و asset محلی توصیه می‌شود.

پایین — persistence چندپردازه

Lock حافظه فقط داخل یک process معتبر است. چند worker می‌توانند history.json را همزمان بازنویسی کنند. برای production از DB یا storage تراکنشی استفاده شود.

راه‌اندازی و استقرار

1. **محیط:** Python 3.11 و virtualenv ایجاد کنید؛ سپس `pip install -r requirements.txt`.
2. **تنظیم secrets:** حداقل `SESSION_SECRET`، `AUTH_USERNAME`، `AUTH_PASSWORD` و در صورت استفاده `AVALAI_API_KEY` را در محیط امن قرار دهید.
3. **مدل محلی:** برای Ollama مدل تنظیم‌شده در `MODEL_NAME` را `pull` و `ollama serve` را اجرا کنید.
4. **داده mock:** یک بار `python db_mock.py`؛ توجه کنید اجرای مجدد DB را بازسازی می‌کند.
5. **اجرا:** `uvicorn app:app --host 0.0.0.0 --port 8000` و سپس بازکردن route ریشه.
6. **استقرار:** Dockerfile برای container و `passenger_wsgi.py` برای cPanel دو مسیر مستقل استقرار هستند.

فهرست کامل فایل‌های بررسی‌شده

ردیف	مسیر	نوع	اندازه
1	.gitignore	متنی/کد	28 خط
2	.test_cookies.txt	متنی/کد	4 خط
3	Dockerfile	متنی/کد	17 خط
4	README.md	متنی/کد	178 خط
5	Rayamate_Presentation.pptx	دودویی	46,614 بایت
6	app.py	متنی/کد	557 خط
7	auth.py	متنی/کد	56 خط
8	avalai_client.py	متنی/کد	141 خط
9	create_presentation.py	متنی/کد	436 خط
10	database.py	متنی/کد	52 خط
11	db_mock.py	متنی/کد	391 خط
12	index.html	متنی/کد	35 خط
13	memory_manager.py	متنی/کد	149 خط
14	ollama_client.py	متنی/کد	243 خط
15	passenger_wsgi.py	متنی/کد	20 خط
16	requirements.txt	متنی/کد	8 خط
17	script.js	متنی/کد	351 خط
18	sql_safety.py	متنی/کد	233 خط
19	static/first.html	متنی/کد	701 خط
20	static/images/login-template.png	دودویی	74,539 بایت
21	static/index.html	متنی/کد	97 خط

ردیف	مسیر	نوع	اندازه
22	static/login.html	متنی/کد	629 خط
23	static/login.js	متنی/کد	100 خط
24	static/script.js	متنی/کد	1,110 خط
25	static/style.css	متنی/کد	1,476 خط
26	static/users.html	متنی/کد	205 خط
27	static/users.js	متنی/کد	179 خط
28	static/vendor/chart.umd.min.js	متنی/کد	20 خط
29	style.css	متنی/کد	426 خط
30	users.py	متنی/کد	245 خط

جمع‌بندی

برنچ `newDashboard2` یک prototype نسبتاً کامل BI مکالمه‌ای است: صفحه معرفی، ورود، داشبورد، مدیریت کاربران، انتخاب دو مدل، تولید و اصلاح SQL، جدول و نمودار، تحلیل ثانویه و تاریخچه شخصی را یک‌جا دارد.

مهم‌ترین کارهای پیش از production عبارت‌اند از حذف و rotate تمام secret‌های commit شده، اجباری کردن تنظیمات امن، افزودن row-level authorization واقعی، CSRF/rate-limit، استفاده از DB برای parser/timeout، history قوی‌تر SQL، همگام‌سازی README و حذف نسخه‌های legacy ریشه.

پایان سند • `commit` {commit_short}